You can configure a wide array of time- and calendar-based triggers with Quartz.NET. You can also control how Quartz.NET handles missed triggers—that is, triggers that should have fired, but your app wasn't running at the time. For a detailed description of the trigger configuration options and more examples, see the Quartz.NET documentation at https://www.quartz-scheduler.net/documentation.

**TIP** A common problem people run into with long-running jobs is that Quartz.NET keeps starting new instances of the job when a trigger fires, even though it's already running. To avoid that, tell Quartz.NET to not start another instance by decorating your `IJob` implementation with the `[DisallowConcurrentExecution]` attribute.

The ability to configure advanced schedules, the simple use of DI in background tasks, and the separation of jobs from triggers are reasons enough for me to recommend Quartz.NET if you have anything more than the most basic background service needs. However, the real tipping point is when you need to scale your application for redundancy or performance reasons; that's when Quartz.NET's clustering capabilities make it shine.

### *34.3.3 Using clustering to add redundancy to your background tasks*

In this section you'll learn how to configure Quartz.NET to persist its configuration to a database. This is a necessary step in enabling clustering so that multiple instances of your application can coordinate to run your Quartz.NET jobs.

As your applications become more popular, you may need to run more instances of your app to handle the traffic they receive. If you keep your ASP.NET Core applications stateless, the process of scaling is relatively